

# Pandrosion pur vectorisé par pentes télescopiques exactes

## Étude mathématique et expérimentale du script 115

Ivan Besevic

29 avril 2026

### Abstract

Nous étudions le moteur `115_pandrosion_vectorized_pure_thales_engine.py`. Contrairement au moteur 113, qui utilise un correcteur local de type Jacobien, 115 construit une matrice de pente exacte  $Q(a, b)$  satisfaisant

$$F(b) - F(a) = Q(a, b)(b - a)$$

par identité télescopique monomiale. Cette construction est derivative-free au niveau du correcteur: aucun Jacobien n'est construit pour la correction locale. Le coût de  $Q(a, b)$  est réduit par vectorisation: puissances tabulées, facteurs préfixe/suffixe et produit matriciel NumPy/BLAS. Le moteur conserve le même flot géométrique unique que 113: Riemann-Möbius, homothétie de type Thalès, optimisation du point initial, puis correction. Sur les tests disponibles, 115 retrouve 144/144 racines pour  $ks(2, 12)$ , 196/196 pour  $ks(2, 14)$ , 64/64 pour  $ks(3, 4)$ , 81/81 pour  $ks(4, 3)$ , et une racine stricte pour  $ks(6, 16)$ . Par rapport à 114, la vectorisation accélère la construction de  $Q$  d'environ  $6\times$  à  $7\times$  sur les cas 2, 12 et 2, 14.

## 1 Motivation

La version 113 a montré qu'un flot géométrique puissant peut extraire toutes les racines de certains systèmes de Kostlan sans suivre explicitement tous les chemins homotopiques. Mais son correcteur local résout

$$J_G(y)\Delta = -G(y).$$

La version 115 remplace cette étape par une pente exacte de Pandrosion. Le but est de conserver l'efficacité globale du flot Thalès-Riemann tout en rendant la correction locale fidèle à l'identité géométrique

$$G(b) - G(a) = Q_G(a, b)(b - a).$$

## 2 Systèmes polynomiaux et changement de coordonnées

Soit

$$F = (F_1, \dots, F_n) : \mathbb{C}^n \rightarrow \mathbb{C}^n$$

un système polynomial dense. Le moteur utilise une carte linéaire

$$z = Ay$$

et résout

$$G(y) = F(Ay) = 0.$$

Comme dans 113, le point initial est généré par un flot unique:

$$y_j = \Lambda p_j \frac{\cos(\theta_j)(u_j/p_j) + \sin(\theta_j)}{-\sin(\theta_j)(u_j/p_j) + \cos(\theta_j)}. \quad (1)$$

Cette carte mélange homothétie, inversion projective et rotation de Riemann dans une seule formule.

### 3 Matrice de pente Pandrosion

**Définition 1.** Soient  $a, b \in \mathbb{C}^n$ . Une matrice  $Q_F(a, b) \in \mathbb{C}^{n \times n}$  est une pente de Pandrosion pour  $F$  entre  $a$  et  $b$  si

$$F(b) - F(a) = Q_F(a, b)(b - a). \quad (2)$$

Pour le système transformé  $G(y) = F(Ay)$ , on pose

$$Q_G(a_y, b_y) = Q_F(Aa_y, Ab_y)A. \quad (3)$$

Alors

$$G(b_y) - G(a_y) = Q_G(a_y, b_y)(b_y - a_y).$$

### 4 Identité télescopique monomiale

Considérons un monôme

$$m(z) = z^\alpha = \prod_{k=1}^n z_k^{\alpha_k}.$$

Pour chaque coordonnée  $j$ , définissons

$$S_{j, \alpha_j}(a_j, b_j) = \begin{cases} \sum_{r=0}^{\alpha_j-1} b_j^{\alpha_j-1-r} a_j^r, & \alpha_j > 0, \\ 0, & \alpha_j = 0. \end{cases} \quad (4)$$

**Lemme 1** (pente télescopique d'un monôme). On a

$$m(b) - m(a) = \sum_{j=1}^n \left( \prod_{k < j} b_k^{\alpha_k} \right) S_{j, \alpha_j}(a_j, b_j) \left( \prod_{k > j} a_k^{\alpha_k} \right) (b_j - a_j). \quad (5)$$

*Proof.* On écrit la différence totale en remplaçant successivement les coordonnées de  $a$  par celles de  $b$ :

$$\prod_k b_k^{\alpha_k} - \prod_k a_k^{\alpha_k} = \sum_j \left( \prod_{k < j} b_k^{\alpha_k} \right) (b_j^{\alpha_j} - a_j^{\alpha_j}) \left( \prod_{k > j} a_k^{\alpha_k} \right).$$

Or

$$b_j^{\alpha_j} - a_j^{\alpha_j} = (b_j - a_j) \sum_{r=0}^{\alpha_j-1} b_j^{\alpha_j-1-r} a_j^r.$$

En substituant cette égalité, on obtient (5). □

Pour un polynôme

$$F_i(z) = \sum_{\alpha} c_{i, \alpha} z^\alpha,$$

la colonne  $j$  de  $Q_F(a, b)$  est obtenue en sommant les contributions (5) pondérées par  $c_{i, \alpha}$ . Cette construction est exacte, à l'erreur flottante près, et ne requiert aucune dérivée.

## 5 Correcteur Pandrosion pur

À l'itération courante  $a$ , le moteur construit un second point  $b$  par une formule géométrique non infinitésimale. Il calcule ensuite  $Q_G(a, b)$  et résout

$$Q_G(a, b)\Delta = -G(a), \quad a_+ = a + \lambda\Delta. \quad (6)$$

Le paramètre  $\lambda$  est choisi par une recherche proportionnelle qui exige une diminution du résidu. Il n'y a pas de fallback Newton, pas de moindres carrés, pas de spécialisation bivariée et pas d'homotopie de secours.

**Remarque 1.** *Le moteur utilise encore `np.linalg.solve`, mais seulement pour résoudre le système linéaire (6). Il ne s'agit pas d'un solveur de racines caché.*

## 6 Vectorisation de $Q(a, b)$

La version 114 construisait  $Q(a, b)$  par boucles Python. Le moteur 115 vectorise le calcul.

Soit  $E \in \mathbb{N}^{M \times n}$  la matrice des exposants des  $M$  monômes, et soit  $C \in \mathbb{C}^{n \times M}$  la matrice des coefficients. Pour des points  $a, b$ , on tabule

$$a_j^m, quad b_j^m, \quad m = 0, \dots, d.$$

Puis on construit pour tous les monômes:

$$P_j^<(b) = \prod_{k < j} b_k^{E_{\cdot k}}, \quad (7)$$

$$P_j^>(a) = \prod_{k > j} a_k^{E_{\cdot k}}, \quad (8)$$

$$T_j = P_j^<(b) P_j^>(a) S_{j, E_{\cdot j}}(a_j, b_j). \quad (9)$$

La matrice de pente est alors obtenue en un produit dense:

$$Q_F(a, b) = CT. \quad (10)$$

La table  $S_{j, m}$  est calculée par la récurrence

$$S_{j, 0} = 0, \quad S_{j, m} = b_j^{m-1} + a_j S_{j, m-1}. \quad (11)$$

Cette récurrence est équivalente à (4).

## 7 Algorithme

**Entrée:**  $F$ , un budget d'essais  $P$ , un nombre cible  $N$ .

**Sortie:** racines numériques distinctes.

1. Générer le système dense et les matrices  $E, C$ .
2. Pour chaque essai, produire  $u, \Lambda, \theta, p$ , puis  $y$  par (1).
3. Améliorer le point de départ par startopt.
4. Répéter:
  - (a) choisir un point géométrique  $b$  associé à  $a$ ;
  - (b) construire  $Q_G(a, b)$  par (3) et (10);
  - (c) résoudre  $Q_G(a, b)\Delta = -G(a)$ ;
  - (d) appliquer une recherche proportionnelle sur  $a + \lambda\Delta$ .
5. Convertir  $z = Ay$ , valider  $\|F(z)\|$ , puis dédupliquer.

## 8 Résultats expérimentaux

| Cas         | Bézout     | Racines demandées | Racines uniques | Essais | Échecs | Temps (s) |
|-------------|------------|-------------------|-----------------|--------|--------|-----------|
| $ks(2, 12)$ | 144        | 144               | 144             | 272    | 124    | 5.345     |
| $ks(2, 14)$ | 196        | 196               | 196             | 390    | 192    | 5.967     |
| $ks(3, 4)$  | 64         | 64                | 64              | 125    | 20     | 1.586     |
| $ks(4, 3)$  | 81         | 81                | 81              | 220    | 57     | 3.464     |
| $ks(6, 16)$ | 16,777,216 | 1                 | 1               | 1      | 0      | 2.639     |

Table 1: Résultats principaux du moteur 115.

| Cas         | Appels $Q$ | Temps $Q$ (s) | Résidu max.           | Résidu moyen          |
|-------------|------------|---------------|-----------------------|-----------------------|
| $ks(2, 12)$ | 6,287      | 1.024         | $9.77 \cdot 10^{-9}$  | $3.79 \cdot 10^{-9}$  |
| $ks(2, 14)$ | 8,624      | 1.154         | $9.96 \cdot 10^{-9}$  | $3.99 \cdot 10^{-9}$  |
| $ks(3, 4)$  | 2,046      | 0.263         | $9.91 \cdot 10^{-9}$  | $4.21 \cdot 10^{-9}$  |
| $ks(4, 3)$  | 3,950      | 0.655         | $9.86 \cdot 10^{-9}$  | $4.17 \cdot 10^{-9}$  |
| $ks(6, 16)$ | 16         | 1.670         | $6.15 \cdot 10^{-13}$ | $6.15 \cdot 10^{-13}$ |

Table 2: Statistiques de pente et résidus pour 115.

## 9 Comparaison avec 114

La version 114 utilisait la même idée de pente télescopique, mais avec construction moins vectorisée. Les mesures disponibles donnent:

| Cas         | Temps 114 (s) | Temps 115 (s) | Accélération totale | Accélération de $Q$ |
|-------------|---------------|---------------|---------------------|---------------------|
| $ks(2, 12)$ | 11.468        | 5.230         | $2.19\times$        | $6.02\times$        |
| $ks(2, 14)$ | 17.457        | 6.735         | $2.59\times$        | $7.43\times$        |

Table 3: Effet de la vectorisation par rapport à 114.

## 10 Discussion

La vectorisation ne change pas le contenu mathématique de la pente. Elle déplace simplement la somme monomiale vers des opérations de type produit matrice-matrice. Ainsi, 115 conserve le caractère Pandrosion pur de 114 tout en réduisant nettement le coût du calcul de  $Q$ . Le coût reste néanmoins plus élevé qu'un correcteur Jacobien classique sur petits degrés, car  $Q(a, b)$  encode une pente globale entre deux points et non seulement une linéarisation infinitésimale.

## 11 Reproductibilité

Commandes principales:

```
python -S 115_pandrosion_vectorized_pure_thales_engine.py \
  --cases 2,12 --equation-normalize --count 144 --pool 5000 \
  --epochs 30 --accept 1e-8 --cluster-sep 1e-8 \
```

```

--startopt-candidates 12 --probe-scale 0.03 --line-search 14

python -S 115_pandrosion_vectorized_pure_thales_engine.py \
  --cases 2,14 --equation-normalize --count 196 --pool 7000 \
  --epochs 30 --accept 1e-8 --cluster-sep 1e-8 \
  --startopt-candidates 12 --probe-scale 0.03 --line-search 14

```

## 12 Conclusion

Le moteur 115 fournit une version autonome, multivariée et derivative-free au niveau du correcteur local de Pandrosion. Sa contribution mathématique est la mise en oeuvre vectorisée de la pente exacte  $Q(a, b)$  issue de l'identité télescopique des monômes. Les résultats montrent que l'on conserve la couverture de racines obtenue par le moteur pur 114 tout en accélérant substantiellement le calcul. Le moteur 115 constitue donc une base plus fidèle à la géométrie Pandrosion qu'un correcteur Newton, tout en restant suffisamment rapide pour des tests denses multivariés.